

BẢN PHÂN TÍCH VÀ GIẢI THÍCH CHI TIẾT DỰ ÁN SPRING BOOT (STORE)

Tài liệu này cung cấp một cái nhìn chuyên sâu, phân tích tỉ mỉ từng góc ngách của dự án Store (Cửa hàng linh kiện máy tính). Mục tiêu là giúp bạn hiểu rõ từng đoạn code, cấu trúc và luồng dữ liệu của toàn bộ dự án.

1. Cấu Trúc Tổng Quan Dự Án

Dự án áp dụng mô hình **MVC (Model - View - Controller)** kết hợp với kiến trúc nhiều lớp (N-Tier Architecture) đặc trưng của Spring Boot.

1.1. Các Gói (Packages) Chính trong `src/main/java/com/codegym/store/` :

- model (Lớp Thực Thể - Entity):** Nơi định nghĩa các đối tượng (ví dụ: `Product`, `Cpu`, `User`, `Cart`). Nhờ Spring Data JPA, các class này sẽ tự động ánh xạ (map) thành các bảng (table) trong database MySQL.
- repository (Lớp Truy Xuất Dữ Liệu - Data Access Layer):** Chứa các interface kế thừa từ `JpaRepository`. Layer này chịu trách nhiệm "nói chuyện" trực tiếp với Database (thực hiện các câu lệnh `SELECT`, `INSERT`, `UPDATE`, `DELETE`).
- service (Lớp Xử Lý Nghiệp Vụ - Business Logic Layer):** Đứng giữa Controller và Repository. Nó tiếp nhận dữ liệu từ Controller, áp dụng các quy tắc kinh doanh (ví dụ: kiểm tra tính hợp lệ, tính toán tổng tiền), rồi mới nhờ Repository lưu xuống Database.
- controller (Lớp Điều Khiển - Presentation Layer):** Bộ não điều hướng web. Lớp này lắng nghe các HTTP Request từ trình duyệt (như `GET /admin/cpu`, `POST /admin/cpu/add`), gọi Service xử lý, rồi nhét kết quả vào Model để hiển thị ra View (giao diện).
- dto (Data Transfer Object):** (Tùy chọn) Đối tượng trung gian dùng để hứng dữ liệu từ Form hoặc đẩy dữ liệu dạng JSON cho API mà không làm lộ các Entity thật.
- config (Lớp Cấu Hình):** Chứa các thiết lập bảo mật (Spring Security), cấu hình tải file, hoặc cấu hình WebMvc.

1.2. Thư Mục Tài Nguyên (`src/main/resources/`):

- templates/** : Chứa toàn bộ giao diện HTML kết hợp cú pháp Thymeleaf (`.html`). Được tổ chức thành các thư mục con như `admin/`, `cpu/`, `store/`.
- static/** : Chứa CSS, JavaScript, thư viện (Bootstrap, jQuery) và hình ảnh tĩnh.
- application.properties** : Trá tìm cấu hình của dự án (port, database url, hibernate ddl-auto).

2. Giải Thích Chi Tiết Từng Thành Phần (Với Code Minh Họa)

2.1. Lớp Model (Định Nghĩa Bảng)

Khác với việc viết SQL tạo bảng thủ công, dự án dùng Hibernate (JPA).

Lớp `Product.java` (Lớp cha)

```
@Entity
@Table(name = "product")
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```

    private Long id;

    @NotBlank(message = "Tên không được trống")
    private String name;

    // ... các trường price, stock, description
}

```

- `@Entity` & `@Table(name = "product")` : Báo cho Spring Boot tạo bảng `product` trong MySQL.
- `@Inheritance(strategy = InheritanceType.JOINED)` : Cơ chế Kế thừa. Vì CPU, RAM, VGA đều là Sản phẩm, nên chúng ta gom các trường chung (Tên, Giá) vào bảng `product` . Các bảng con như `cpu` sẽ chỉ lưu các trường riêng (socket, số nhân) và liên kết với bảng `product` bằng ID.
- `@Id` & `@GeneratedValue(strategy = GenerationType.IDENTITY)` : Định nghĩa khóa chính (Primary Key) tự tăng.

Lớp `Cpu.java` (Lớp con)

```

@Entity
@Table(name = "cpu")
public class Cpu extends Product {
    private Integer cores;
    private Integer threads;
    private String socket;
}

```

- Lớp này kế thừa `Product` . Trong Database, sẽ tạo ra bảng `cpu` liên kết khóa ngoại với bảng `product` .

2.2. Lớp Repository (Thao Tác Database)

`CpuRepository.java`

```

public interface CpuRepository extends JpaRepository<Cpu, Long> {
}

```

- Cực kỳ kỳ diệu: Bạn chỉ cần viết 2 dòng code. `JpaRepository<Cpu, Long>` có nghĩa là kho dữ liệu dành cho đối tượng `Cpu` , với ID kiểu `Long` .
- Ngay lập tức, bạn có thể gọi `cpuRepository.save()` , `cpuRepository.findById()` , `cpuRepository.findAll()` từ Controller mà không cần viết lệnh SQL nào.

2.3. Lớp Controller (Bộ Điều Phối)

Ví dụ: `HomeController.java`

```

@Controller
public class HomeController {
    private final ProductRepository productRepository;

    public HomeController(ProductRepository productRepository) {
        this.productRepository = productRepository; // Tiêm (Inject) kho dữ liệu vào
    }
}

```

```

@GetMapping("/")
public String home(@RequestParam(defaultValue = "0") int page, Model model) {
    // 1. Phân trang: Lấy trang 'page', mỗi trang 50 sản phẩm
    Pageable pageable = PageRequest.of(page, 50);

    // 2. Trích xuất dữ liệu từ DB
    Page<Product> productPage = productRepository.findAll(pageable);

    // 3. Đóng gói dữ liệu mang qua cho HTML
    model.addAttribute("cpus", productPage.getContent());
    model.addAttribute("currentPage", page);
    model.addAttribute("totalPages", productPage.getTotalPages());

    // 4. Chỉ định file HTML để render: src/main/resources/templates/store/home.html
    return "store/home";
}
}

```

- `@GetMapping("/")` : Khi User vào trang chủ `http://localhost:8080/` , hàm này sẽ chạy.
- Tham số `Model model` : Là "cái túi" chứa dữ liệu để mang từ Java sang HTML.
`model.addAttribute("tên_biến", giá_trị)` là cách bỏ dữ liệu vào túi.

2.4. HTML và Thymeleaf (Hiển Thị Giao Diện)

Ví dụ: `list-cpu.html`

```

<tbody>
    <!-- Vòng lặp: Lặp qua danh sách 'cpus' được gửi từ Controller -->
    <tr th:each="cpu : ${cpus}">
        <td th:text="${cpu.id}">1</td>
        <td th:text="${cpu.name}">Core i9</td>

        <!-- Hiển thị ảnh đầu tiên của sản phẩm nếu có -->
        <td>
            
        </td>
    </tr>
</tbody>

```

- `th:each` : Tạo vòng lặp (giống `for` trong Java).
- `th:text` : In giá trị của biến ra thay thế cho chữ (text) của thẻ HTML.
- `${...}` : Cú pháp lấy giá trị của biến nằm trong túi `Model` mà Controller vừa nhét vào.
- `@^{...}` : Dùng để trỏ đường dẫn (link, src) một cách an toàn so với URL gốc của web.

2.5. JavaScript (JS - Xử Lý Client Side)

Trong các file `.html` hoặc `.js` , JS đảm nhận các việc:

- **Xác nhận xóa:**

```
function confirmDelete(id) {  
    if (confirm("Bạn có chắc chắn muốn xóa sản phẩm này không?")) {  
        window.location.href = "/admin/cpu/delete/" + id;  
    }  
}
```

- **Tải ảnh Preview (xem trước):** Dùng `FileReader` API để hiển thị ảnh ngay lập tức khi User chọn ảnh từ máy tính (chưa cần ấn nút submit form).
- **AJAX Tìm Kiếm:** Trong `HomeController`, có API trả về mảng JSON. Bằng cách dùng `fetch()` trong JS, trình duyệt có thể hiển thị kết quả tìm kiếm thả xuống (dropdown) mà không cần load lại cả trang.

3. Phân Tích Luồng Dữ Liệu Xuyên Suốt (End-to-End Flow)

Hãy xem luồng "Quản trị viên thêm mới một CPU":

1. Hiển Thị Form (GET):

- User gõ URL `/admin/cpu/add`.
- `CpuController` bắt URL bằng `@GetMapping("/add")`.
- Controller tạo ra một đối tượng `new Cpu()` rỗng, đẩy vào `Model` với tên "product".
- Controller trả về view `cpu/add-cpu.html`.
- Thymeleaf sinh ra HTML form, gán (bind) form này với đối tượng `product`.

2. Người Dùng Nhập Form:

- User nhập tên, giá, số nhân, tải ảnh lên, và bấm "Lưu".
- Dữ liệu được gửi đi dưới dạng HTTP POST (`<form method="post" enctype="multipart/form-data">`).

3. Xử Lý Dữ Liệu (POST):

- `CpuController` bắt dữ liệu bằng `@PostMapping("/add")`.
- Spring Boot tự động "nhào nặn" các field từ Form để nhét lại vào object `Cpu` (thông qua `@ModelAttribute`).
- Nếu có file ảnh, Controller gọi `StorageService` để lưu file vào ổ cứng (thư mục `uploads/`), lấy đường dẫn file (path).
- Khởi tạo `ProductImage` (lưu đường dẫn) và thêm vào danh sách ảnh của đối tượng `Cpu`.
- Kích hoạt `productService.save(cpu)`.
- `ProductService` nhờ `CpuRepository.save()`.
- Hibernate "dịch" toàn bộ việc này thành câu lệnh: `INSERT INTO product... , INSERT INTO cpu... , INSERT INTO product_image...` (do có `CascadeType.ALL` nên nó tự động lưu ảnh theo).

4. Kết Thúc:

- Controller chạy lệnh `return "redirect:/admin/cpu";`.
- Trình duyệt tự động chuyển hướng về trang danh sách.

4. File Cấu Hình (`application.properties`)

```
spring.application.name=store
# Khai báo đường dẫn CSDL MySQL, tài khoản, mật khẩu
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/store}
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD:root@123}

# Hibernate sẽ TỰ ĐỘNG CẬP NHẬT CẤU TRÚC BẢNG (thêm cột, tạo bảng mới) nếu Code thay đổi.
spring.jpa.hibernate.ddl-auto=update

# Hiển thị lệnh SQL thực tế chạy dưới nền (tiện cho debug)
spring.jpa.show-sql=true
```

- Các biến có dạng `${DB_URL:...}` nghĩa là dự án có thể lấy thông tin từ "Biến môi trường" (Environment Variables) nếu ứng dụng được đưa lên các dịch vụ đám mây (Render, AWS), còn nếu chạy ở máy tính cá nhân (localhost) thì nó sẽ lấy giá trị mặc định đăng sau dấu `:`.

Tổng Kết: Sự kết hợp hoàn hảo giữa Spring Boot (chuyên lo Backend), Hibernate (chuyên lo Database), và Thymeleaf + HTML/JS (chuyên lo Frontend) tạo nên một hệ sinh thái Web chặt chẽ, dễ bảo trì và dễ dàng mở rộng.